Week 3: Data Structures & the Supply Chain Network

Welcome to Week 3.

Last week you learned control flow and functions. This week, you'll learn how to organise data using Python's built-in data structures: lists, tuples, dictionaries and sets.

You'll use these structures to understand the supply chain as a network: which suppliers provide which product categories, where the company depends on a single supplier, and how spend is spread across suppliers.

Why this matters: Real data is connected. Products connect to suppliers, and orders connect to both. Knowing how to structure data is what makes it possible to analyse it.

Project Goal

Use lists, dictionaries and sets to organise the supply chain data into meaningful structures, and produce a supply risk report.

Part A: What You Will Learn

Essential Concept 1: Lists

What they are: Ordered collections of items, e.g. `costs = [6062.06, 1226.08, 53976.5]`.

Why you need them: To store many related values and access them by position.

Essential Concept 2: List Methods, Indexing and Slicing

What they are: Actions such as `append()` and `sort()`, and ways to reach items: `costs[0]` (first), `costs[-1]` (last), `costs[:5]` (first five).

Essential Concept 3: Tuples

What they are: Like lists, but cannot be changed once created, e.g. `point = (3, 4)`.

Essential Concept 4: Dictionaries

What they are: Collections of key-value pairs, e.g. `{"SupplierA": "USA"}`.

Why you need them: To look data up by a meaningful label instead of a position. Each row from `csv.DictReader` is a dictionary.

Essential Concept 5: Sets

What they are: Unordered collections of unique items.

Why you need them: To find unique values, and to compare groups with set operations: `a & b` (in both), `a | b` (in either), `a - b` (in a but not b).

Essential Concept 6: Nested Data Structures

What they are: Structures inside structures, e.g. a dictionary whose values are sets or other dictionaries.

Why you need them: To represent real relationships, such as "each supplier, the categories it supplies and its total spend".

Part B: Video Resources

Video	Topic	Link	Focus
1	Lists, Sets and Tuples, Bro Code (15 min)	Watch	Creating and using the three collection types
2	Slicing Lists and Strings, Corey Schafer (11 min)	Watch	Indexing and slicing
3	Dictionaries, Corey Schafer (10 min)	Watch	Key-value pairs and lookups
4	Sets: Methods and Operations, Corey Schafer (19 min)	Watch	Union, intersection, difference
5	Dictionaries + Lists of Dicts (8 min)	Watch	The structure `csv.DictReader` gives you

---

Part C: The Tasks

Load the dataset as you did in Week 2. Last week you saw that supplier names are written in 15 different ways and some categories have extra spaces. Until you learn to clean data properly in Week 4, use these two helper functions whenever you read a supplier name or a category:

```
def clean_supplier(name):
    """'Supplier A', 'suppliera', 'SupplierA' -> 'SupplierA'"""
    compact = name.strip().lower().replace(" ", "")
    return "Supplier" + compact[-1].upper()

def clean_category(category):
    """' Motors ' -> 'Motors'"""
    return category.strip()
```

Task 1: Organise Orders in a List

Create a list of every order's `total_cost` (as numbers). Then:

- Find the total spend and the average order value

- Sort the list and find the median (the middle value; with an even number of orders, average the two middle values)
- Print the five smallest and five largest orders

Task 2: Map Categories to Suppliers

Build a dictionary where each key is a product category and each value is a **set** of the suppliers that provide it. (A set, not a single name: some categories have more than one supplier.)

- Print each category with its suppliers
- Look up who supplies "Pumps"
- Count how many unique suppliers there are

Task 3: Find Unique Values with Sets

Create sets for supplier names, product categories and delivery statuses. For each, print how many unique values exist **before** and **after** using the helper functions. Which statuses look like typos?

Then use set operations on your Task 2 dictionary:

- Which suppliers supply both Motors and Pumps?
- Which supply Motors or Pumps?
- Which supply Motors but not Pumps?

Task 4: Single-Source Risk Report

If a category has only one supplier and that supplier fails, the company cannot get that product. This is called single-source risk.

Loop through your Task 2 dictionary and print a risk report listing every category that depends on only one supplier, and which supplier it is. End with a line such as "4 of 8 categories are single-source".

Task 5: Build the Supplier Network

Create a nested dictionary with one entry per supplier, for example:

```
network["SupplierA"] = {"categories": set(),   # categories it supplies
                        "order_count": 0,      # add 1 for each order
                        "total_spend": 0.0}    # add each order's total_cost
```

Then query it: show everything about SupplierA, and print one line per supplier with its order count, number of categories and total spend.

Extra Challenges

Challenge 1: Make a list of orders above 100,000. What percentage of all orders is that?

Challenge 2: Which supplier provides the most categories?

Challenge 3: Build a dictionary that groups order costs by delivery status. What is the average value of a "Delivered" order?

What You've Learned This Week

- Lists: creation, indexing, slicing, sorting
- Dictionaries: key-value pairs and lookups
- Sets: unique values and set operations
- Nested data structures
- Spotting supply risk with code

Common Mistakes

Mistake 1: Using a position that doesn't exist

```
orders = [100, 200, 300]
print(orders[3])   # IndexError: positions are 0, 1 and 2
print(orders[-1])  # RIGHT: the last item
```

Mistake 2: Looking up a key that doesn't exist

```
suppliers = {"Pumps": {"SupplierC"}}
print(suppliers["Fans"])          # KeyError
print("Fans" in suppliers)        # RIGHT: check first (False)
```

Mistake 3: Overwriting dictionary values

```
# WRONG: each new supplier replaces the last one
category_suppliers[category] = supplier

# RIGHT: keep all of them in a set
category_suppliers[category].add(supplier)
```

Mistake 4: Expecting duplicates in a set

```
items = {"SupplierA", "SupplierA", "SupplierB"}
print(len(items))   # 2, not 3: sets keep each value once
```

Mistake 5: Assigning the result of sort()

```
numbers = [3, 1, 2]
numbers = numbers.sort()   # WRONG: sort() returns None
numbers.sort()             # RIGHT: sorts the list in place
sorted_copy = sorted(numbers)   # or get a new sorted list
```

Part D: Deliverables for Week 3

1. Your notebook with all 5 tasks, clean comments and no errors, uploaded to GitHub.
2. A Supply Chain Network Summary (one paragraph):
    - How many real suppliers and categories are there?
    - Which categories are single-source, and why is that a risk?
    - Which supplier receives the most spend?

What to post: Screenshots of your risk report and supplier network output.

Part E: Submit Your Work

Submission Period: Friday 7am to Saturday 5pm (WAT)

Where to Submit: X (Twitter) or LinkedIn (or both!), plus the submission form that the Group Admin will drop in the WhatsApp group. Paste the link to your GitHub repository in the form.

For X (Twitter) submissions, please tag:

1. @TDataImmersed (TDI Official page)
2. @DabereNnamani (Annie)
3. @ThePSF
4. Use the hashtag #TheDataImmersed

For LinkedIn submissions, please tag:

1. TDI page
2. Annie Nnamani

Next week: File handling and a full data quality check of the dataset!